Understanding RealTOA

A Telegram Bot that automates tracking mileage for SAF Transport Operators (TO), thereby becoming a 'Real TO Assistant (RealTOA)'

1 PROBLEM STATEMENT

In the Singapore Armed Forces (SAF), Transport Operators (TOs) are required to log mission mileage to qualify for a civilian license conversion, upon reaching 4000km. The current workflow to detect accumulated mileage relies on:

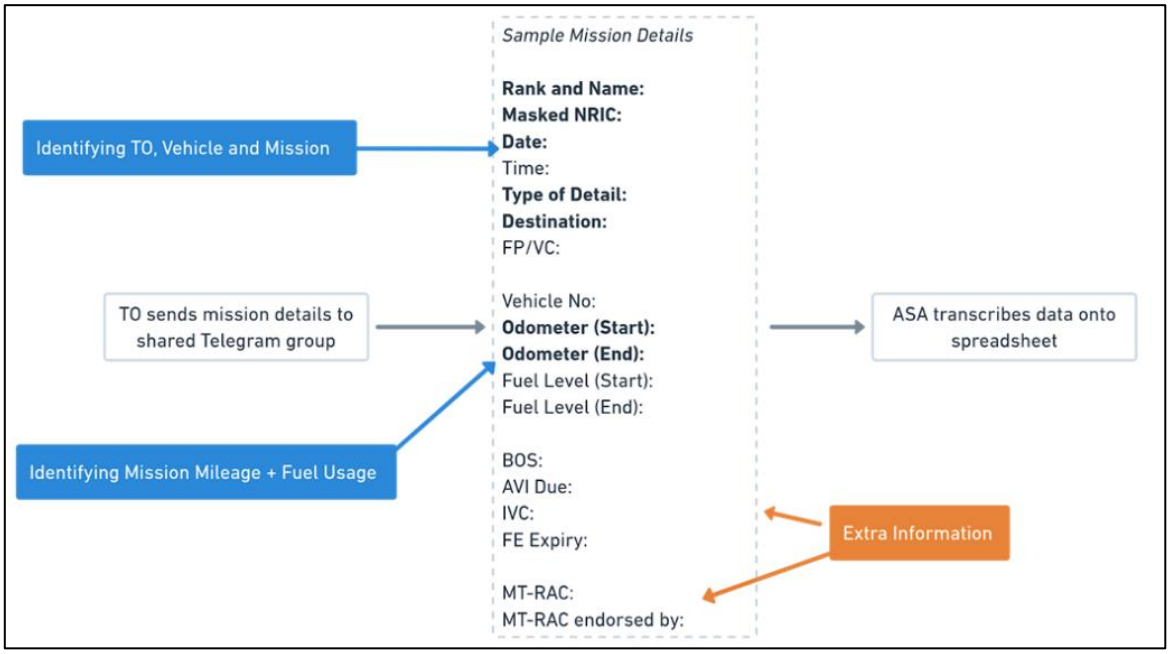

Figure 1: Current Workflow

1. TOs submit mission details on Telegram
2. Manual transcription from Telegram to Spreadsheets by Admin Support Assistants (ASAs).

From this workflow, the following pain points were observed:

Pain Points	Impact
Overlooked messages by ASAs	Entry is not recorded → lost mission mileage records
Incorrect mission details sent by TO/ transcription by ASAs	Inaccurate odometer logs → inaccurate mileage totals (worse if it is lower than actual)
No direct TO visibility of spreadsheet	TOs cannot verify their own records and mileage without requesting the ASAs for access
Numerous TOs → high message volume	Manual work → administrative inefficiencies and delays

Table 1: Identified Failure Points

2 PROJECT OBJECTIVES

To target the pain points observed in Table 1, this project aims to:

- **Reduce Human Error** in mileage recording for both the TOs and ASAs.
- **Improve the efficiency and accuracy of updating** and maintaining the mileage records
- Create a more **transparent and reliable** mileage recording system for TOs
- **Automate the mileage recording system** for ASAs

3 PROPOSED SOLUTION

The proposed system is a **Passive, Parsing Telegram Bot.** It is integrated within the existing Telegram group, and operates by minimally adjusting TOs' and ASAs' existing workflows.

3.1 WORKFLOW DESCRIPTION

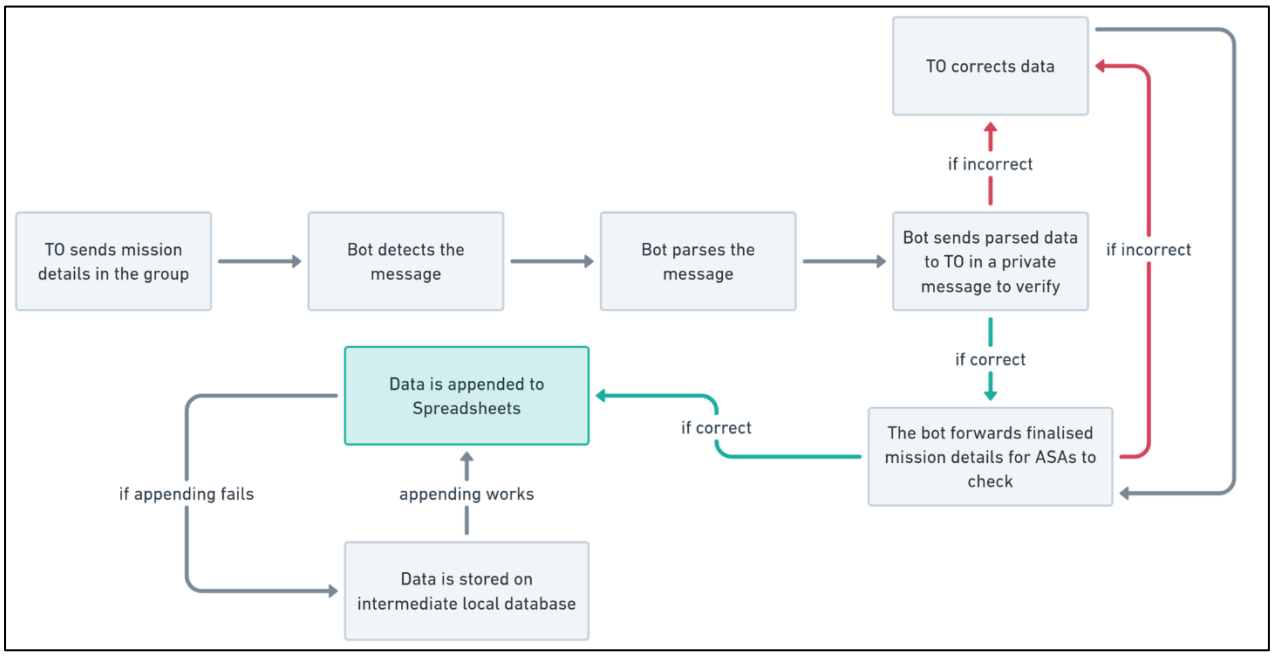

Figure 2: Proposed System Workflow

In short, the Telegram Bot:

1. Monitors group messages in real time
2. Extracts key information from messages, to append onto spreadsheets.
3. Validates and standardises extracted information
4. Requests TO's confirmation (First confirmation loop)
5. Requests ASAs' confirmation (Second confirmation loop)
6. Appends information to spreadsheet (Intermediate storage on database should appending fails)

4 SYSTEM ARCHITECTURE

No	Module	Requirements	Implementation	Edge Cases
1	Message Detection in Telegram	➢ System detects all, relevant text messages in the group, in real time	<u>Detecting all Messages</u> Primary: Webhooks Testing/Backup: Polling <u>Filtering Relevant Messages</u> ➢ Ensure at least 5/19 fields present ○ Name ○ Date ○ Vehicle Number ○ Start Odometer ○ End Odometer	1. <u>High Message Volume by same person</u> ➢ Implement per-user cooldown 2. <u>Irrelevant Messages</u> ➢ Filtered via regex parsing pre-check
2	Parser *Check Appendix A*: Parser proof of concept *for Proof of Concept*	➢ Extract data from Telegram messages ➢ Achieve 100% parsing accuracy	<u>Data Extraction</u> ➢ Regex-based extraction ➢ Removing case-sensitivity ➢ Field mapping common terms with proper ones ○ "start odo" → Odometer (S) <u>Potential Improvements:</u> ➢ Implement ML/NLP parsing to increase accuracy	1. <u>Parsing Failure</u> ➢ Request TO correction (confirmation loop) 2. <u>Missing Fields</u> ➢ Request TO confirmation (confirmation loop)

No	Module	Requirements	Implementation	Edge Cases
3	Data Validation	➢ Ensure all extracted data meets required formats for easy appending. ➢ Detect Duplicate Submissions	<u>Format Validation</u> ➢ Date: DD/MM/YY ➢ Fuel: 0 to 100% ➢ Odometer: o Positive integers o End Odometer ≥ Start Odometer <u>Detect Duplicate Submissions</u> ➢ Cross-reference following fields together: o Date + o Vehicle Number + o Odometer (S) + o Odometer (E)	1. <u>Logical Failure</u> ➢ Request TO correction (confirmation loop) 2. <u>Duplicate Detected</u> ➢ Highlight to ASA (admin role)
4	Data Confirmation	➢ System recognises sender TO, and initiates DM with him. ➢ Require TO to verify all extracted data ➢ Require TO to enable incorrect fields ➢ Require secondary confirmation by ASA before appending to spreadsheet	<u>Initiate DM with Sender TO:</u> ➢ Field Mapping 'masked NRIC' with Telegram 'user ID' <u>Confirmation loops (TO, ASA):</u> ➢ Send parsed data via Telegram DMs ➢ Buttons (Confirm, Edit) ➢ Guided Prompts to edit incorrect fields	1. <u>Bot cannot initiate DM with TO</u> ➢ Notify ASA ➢ Notify TO in Telegram group 2. <u>TO / ASA does not respond</u> ➢ Require bot to send reminder - If TO does not confirm within time-frame, ASA notified. - If ASA does not confirm within time-frame, TO notified.

No	Module	Requirements	Implementation	Edge Cases
5	Data Storage	➢ Ensure all data has been stored ➢ Ensure no data loss during appending failure ➢ Ensure system notifies TO and ASA upon successful data-appending to the spreadsheet	<u>Data Storage</u> Primary: Google Sheets Secondary (if Primary fails): SQLite	1. <u>Google Sheets API Failure</u> ➢ Implement 3 retries before declaring primary method down → notify admin → start secondary storage method ➢ Ensure automated batch upload from SQLite to Google Sheets once API call-function restarts.
6	Bot Crash Resolution	➢ Ensure bot automatically recovers from crashes	<u>Bot Recovery</u> ➢ Cloud-based 'Virtual Private Server' (VPS) Deployment ➢ *Systemd* process manager (restart automatically on crashes) ➢ Logging (to track errors) <u>Future</u> ➢ Docker Containerisation for scalability and deployment to other nodes	1. <u>VPS is down</u> ➢ Temporary local deployment on laptop for recovery till VPS is available

Table 2: System Architecture

*Please check **Figure 3** for implementation status of the different aforementioned modules.*

5 Testing and Deployment

5.1 Testing Criteria

To test, there are 2 main categories to consider:

1. Behavioural Testing
    a. Collect Usability Feedback (both Verbal and Written via Google Forms)
2. Technical Testing
    a. Checking Parsing Accuracy
    b. Testing system under high message volume scenarios
    c. Checking crash-recovery process
    d. Ensuring no data-loss

5.2 Testing Method

Phase 1

- **5-10 TOs**
- 2 Weeks of Testing
- Iterative Improvements

Phase 2

- **25-30 TOs**
- 2 Weeks of Testing
- Iterative Improvements

Phase 3

- **Node-wide Implementation**
- 2 Weeks of Testing
- Iterative Improvements

Phase 4

- Node-wide deployment

Prior to each testing phase, TOs and ASAs will be briefed regarding the bot's usage.

6 ROADMAP

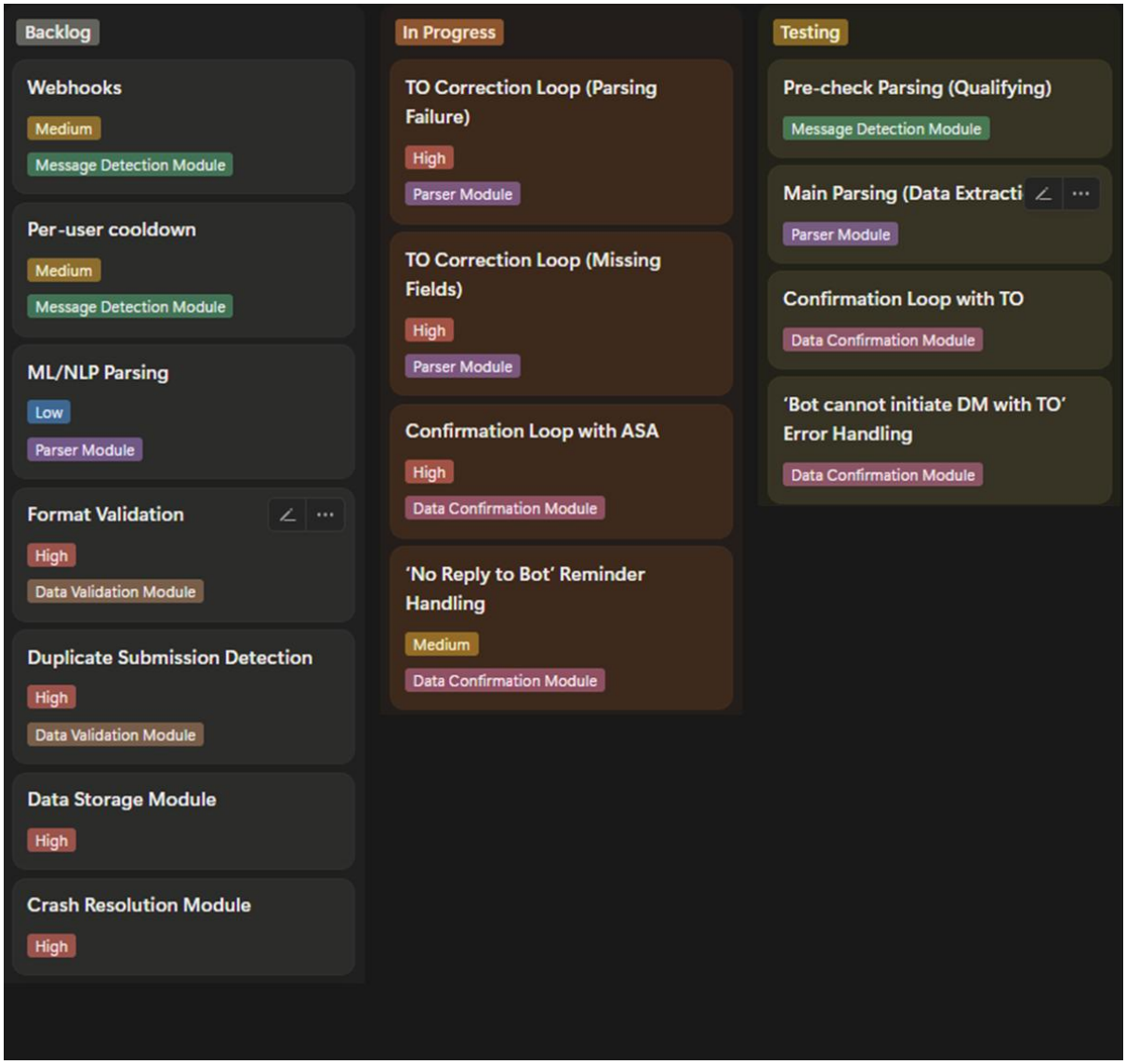

Figure 3: Roadmap of Feature Implementation

7 APPENDIX A: PARSER PROOF OF CONCEPT

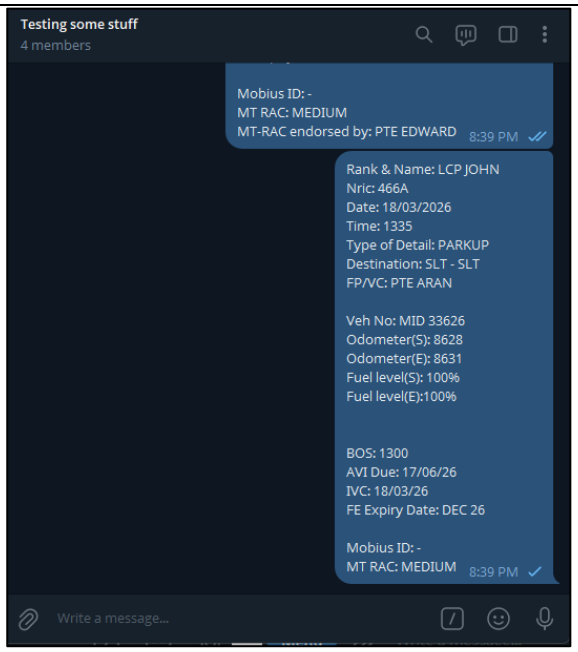

Figure 4: Attempt 1 (Unparsed)

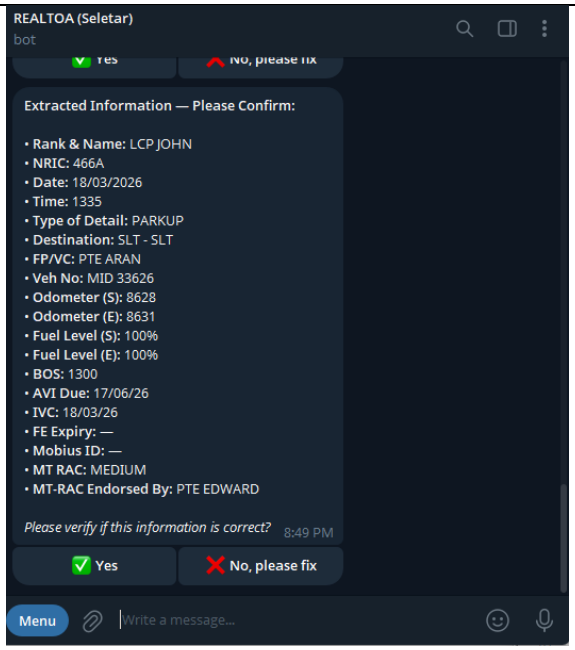

Figure 5: Attempt 1 (Parsed)

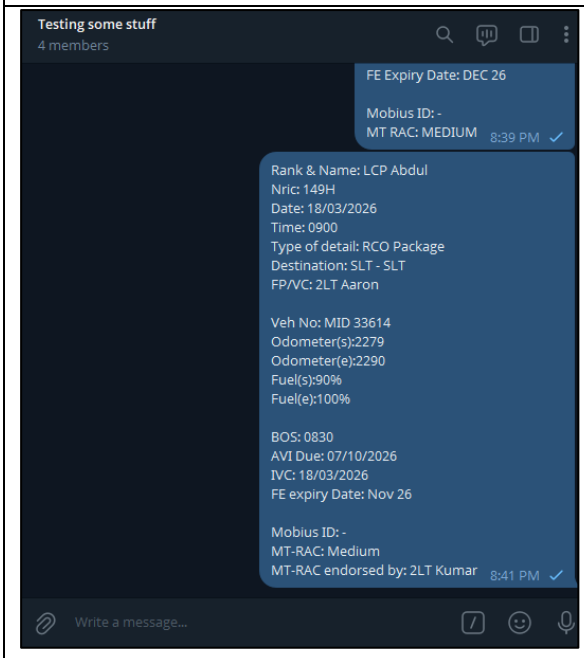

Figure 6: Attempt 2 (Unparsed)

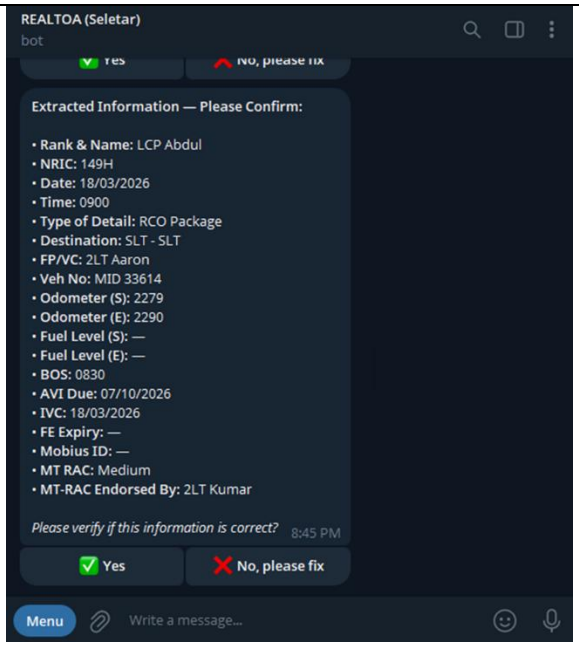

Figure 7: Attempt 2 (Parsed)

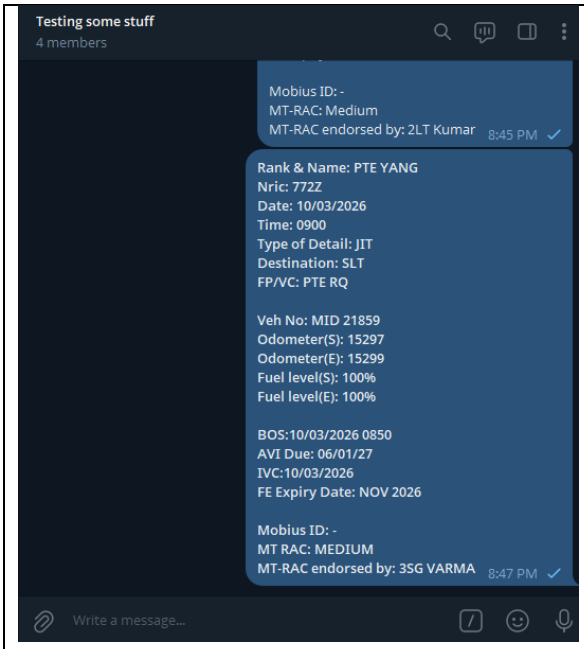

Figure 8: Attempt 3 (Unparsed)

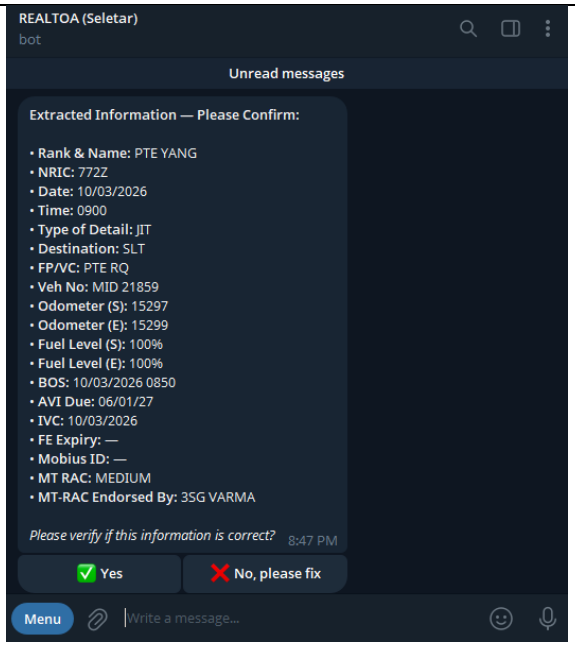

Figure 9: Attempt 3 (Parsed)

8 APPENDIX B: OTHER APPROACHES

8.1 EXTRA APPROACH 1 (EA1) – INTEGRATING 'STRUCTURED-FORMS' INTO TELEGRAM

This approach replicates a structured form-based input system (like Google Forms) within Telegram through a bot interface.

8.1.1 EA1 Workflow Description

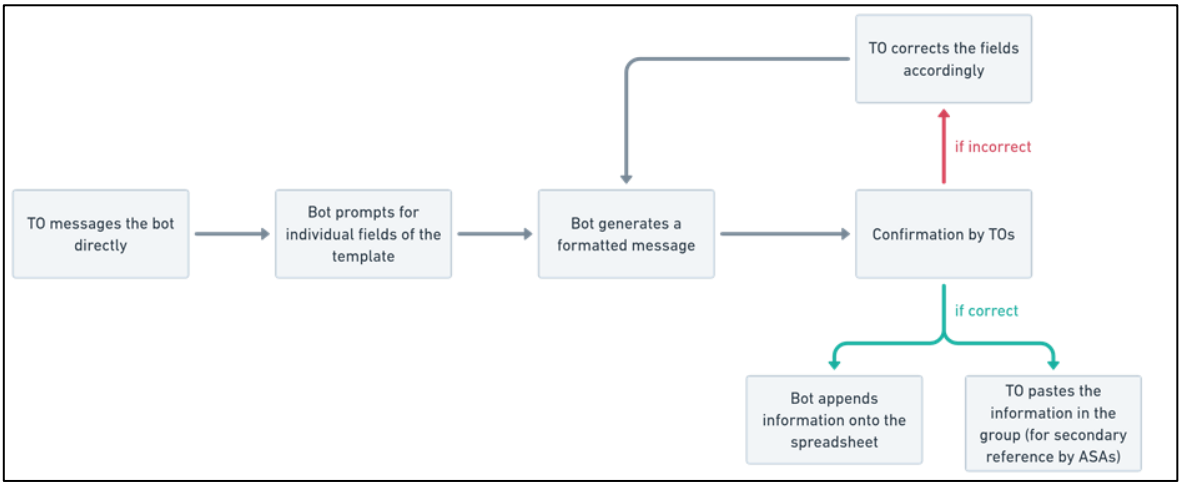

Figure 10: Workflow (EA1)

The workflow operates in the following manner:

1. The TO initiates the process by messaging the bot
2. The bot sequentially prompts the TOs for required fields in the mission template (shown in Figure 1 and Figure 4 to Figure 9).
3. Relevant personal information is pre-filled to reduce redundant effort.
4. The bot compiles all completed fields into a formatted message.
    a. Upon confirmation by the TO, this information is appended to the spreadsheet, where it is verified by the ASAs.
    b. The TO sends this message in the Telegram group (for secondary verification and reference for ASAs).

8.1.2 Evaluating EA1

Advantages	Limitations
Pre-filled fields → reduces TO effort → faster uploads	Numerous sequential prompts → increase interaction time → perceive bot to be bothersome → reduced engagement
No additional software required (everything native in Telegram) → less behavioural friction → higher likelihood of compliance	Introduction of a bot still involves behavioural friction
Confirmation Loop → TOs forced to verify their records → Accurate mileage tracking	Manual copying and pasting of messages from the bot to the telegram group introduces redundant effort.
Logic for appending data to spreadsheet is simple and straightforward.	Requires continuous uptime for bot to be available for 24/7 messaging and appending.
	API costs for appending to spreadsheet

Table 3: Evaluating EA1